

PRUEBAS QUE ATRAPAN ERRORES: CAZAR MUTANTES

Escribir desde cero la suite de un contrato que no trae pruebas, medir su cobertura, demostrar que atrapa tres errores escondidos y desplegarlo verificado en Sepolia con Hardhat Ignition.

MODALIDAD	DURACIÓN	REQUISITO	META
En parejas en la sala	80 min en clase + cierre en casa	Node.js 22 + billetera del curso	≥ 10 pruebas · ≥ 80 % 3 mutantes muertos

CÓMO LEER ESTA GUÍA

Está escrita para hacer el laboratorio **sin ayuda**, desde un computador de la sala recién encendido. Cada paso dice qué escribir, qué deberían ver y cómo saber que funcionó. Las salidas que se muestran son **reales**: se obtuvieron con este mismo repositorio (Hardhat 3.16.0, Solidity 0.8.28). Si algo no coincide, vayan a **16 · Errores frecuentes**.

01 Qué se construye y por qué

[LEER PRIMERO](#)

Objetivo de la sesión (plan del curso): salir del navegador y trabajar con el flujo real de un equipo de desarrollo blockchain. **Resultado de aprendizaje:** RA4 — construir una suite de pruebas automatizadas y desplegar contratos verificados en una red de prueba usando un entorno profesional.

`contracts/s08/Recaudo.sol` es un contrato de recaudo con meta y plazo: un semillero recoge aportes; si al vencer el plazo se alcanzó la meta, el beneficiario reclama los fondos; si no, cada aportante recupera lo suyo. **No trae pruebas: escribirlas es el laboratorio.**

En la misma carpeta hay tres copias — `RecaudoMutante1`, `2` y `3` — con un error sutil cada una. Una suite de pruebas buena:

1. pasa **completa** contra `Recaudo`, y
2. **falla** (al menos una prueba) contra **cada** mutante.

EL DATO QUE MOTIVA TODO EL LABORATORIO

Al preparar el curso se escribió una suite de **13 pruebas** para `Recaudo` que alcanza **88 % de cobertura de líneas** —cumple la meta del curso— y sigue **completamente en verde contra los tres mutantes**. Con tres pruebas de borde más, los tres mueren. La cobertura dice dónde no hay pruebas; no dice que las que hay sirvan.

LA REGLA DEL JUEGO

No abran los archivos de los mutantes hasta que los tres estén muertos. El ejercicio es pensar en los bordes del contrato correcto. Comparar los archivos antes convierte el laboratorio en copiar una diferencia, y lo que se evalúa —y lo que van a necesitar en su proyecto y en el parcial— es el método.

02 Qué se entrega

EVIDENCIA DE LA SESIÓN

La evidencia que pide el plan para la Sesión 8 es **repositorio con suite de pruebas verde y contrato verificado**. En concreto, por pareja:

1. `test/s08/Recaudo.test.js` con **al menos 10 pruebas**, todas en verde contra `Recaudo`.
2. Captura del informe de cobertura con **Line % ≥ 80** en la fila de `contracts\s08\Recaudo.sol`.
3. Salida de `node scripts/s08/cazar-mutantes.js` con los **tres mutantes muertos**, y la tabla de la sección 10.4.
4. Dirección de `Recaudo` en Sepolia, **desplegado con Ignition y verificado**, y el hash de al menos un aporte.
5. Las respuestas de reflexión (sección 17).

03 Preparación desde cero en la sala

10 MINUTOS

3.1 • Abrir una terminal en la carpeta del repositorio

En Windows, abran la carpeta `laboratorios-evm` en el Explorador, hagan clic en la barra de dirección, escriban `powershell` y Enter. En macOS o Linux: `cd` y arrastren la carpeta a la terminal. Comprueben con `dir` (Windows) o `ls`: deben ver `hardhat.config.js` y `package.json`.

3.2 • Comprobar las herramientas

```
node --version      # esperado: v22.x.x
git --version       # esperado: git version 2.x
npx hardhat --version # esperado: 3.16.0
```

Si `node` no se reconoce, instalen Node.js 22 LTS desde `nodejs.org` y abran una terminal nueva. Si Hardhat no responde porque falta `node_modules`, sigan con 3.3.

3.3 • Instalar las dependencias (una vez por semestre)

```
npm install
```

Qué hace: lee `package.json` y `package-lock.json` y descarga exactamente las versiones congeladas del curso (Hardhat 3.16.0, toolbox 3.0.7, OpenZeppelin 5.6.1). **Salida esperada:** termina con `added NNN packages`. En la red de la sala puede tardar varios minutos. Las líneas `npm WARN` no son errores. Si falla por la red, reintenten una vez; si persiste, avisen: el docente tiene una copia del repositorio con `node_modules` lista.

3.4 • La prueba de humo

```
npx hardhat test test/s07/Subasta.test.js
```

Salida esperada: `23 passing`. **Cómo sé que funcionó:** si esto pasa, el compilador, OpenZeppelin, la red local y Mocha funcionan. Si falla, casi siempre es que dejaron su versión incompleta de la S7: `git checkout contracts/s07/Subasta.sol`. La advertencia sobre `Operaciones.sol` («can be restricted to view») es intencional y no es un error.

04 El proyecto por dentro

ruta	qué es
<code>hardhat.config.js</code>	Compilador 0.8.28 con optimizador (200), redes <code>sepolia</code> y <code>opSepolia</code> , verificación. Los secretos aparecen solo como <code>configVariable("NOMBRE")</code> : nunca el valor.
<code>contracts/s08/Recaudo.sol</code>	El contrato a probar.
<code>contracts/s08/RecaudoMutante1..3.sol</code>	Los mutantes. No se abren hasta el final.
<code>andamiaje/s08/Recaudo.test.js</code>	La plantilla de su suite, con dos pruebas de ejemplo y los grupos armados.
<code>scripts/s08/cazar-mutantes.js</code>	Corre su suite contra el correcto y contra los tres mutantes.
<code>ignition/modules/Recaudo.js</code>	Cómo se despliega <code>Recaudo</code> , con parámetros.
<code>ignition/parametros/recaudo-sepolia.json</code>	Los valores para Sepolia: meta 0,005 ETH, plazo 1 800 s (30 min).

05 Leer el contrato como quien busca errores

10 MINUTOS · CON LÁPIZ

Abren `contracts/s08/Recaudo.sol` completo. Tiene un constructor, una consulta y tres funciones que cambian estado:

función	qué hace	errores que puede lanzar
<code>constructor(meta, duracion)</code>	Fija la meta, el fin del plazo y el dueño (quien despliega).	<code>ParametrosInvalidos</code>
<code>estado()</code>	Activo, Exitoso o Fallido, calculado con el tiempo y el total.	—
<code>aportar()</code>	Suma el aporte a la cuenta y al total. Emite <code>Aporte</code> .	<code>PlazoVencido</code> , <code>AporteCero</code>
<code>reclamarFondos()</code>	Solo el dueño, una vez, si la meta se alcanzó. Emite <code>FondosReclamados</code> .	<code>PlazoVigente</code> , <code>MetaNoAlcanzada</code> , <code>YaReclamado</code> , <code>OwnableUnauthorizedAccount</code>
<code>reembolsar()</code>	Cada aportante recupera lo suyo si la meta NO se alcanzó. Emite <code>Reembolso</code> .	<code>PlazoVigente</code> , <code>MetaAlcanzada</code> , <code>NadaQueReembolsar</code>

final	condición, después del plazo	qué se puede hacer
Exitoso	<code>totalRecaudado >= meta</code>	El dueño reclama los fondos, una sola vez.
Fallido	<code>totalRecaudado < meta</code>	Cada aportante recupera exactamente lo suyo.

EJERCICIO · EL INVENTARIO DE LÍMITES (LLÉNELLO ANTES DE ESCRIBIR PRUEBAS)

Por cada comparación (<, <=, >, >=, ==) y cada cambio de estado del contrato, una fila. Esta tabla es su lista de pruebas de borde: la sección 10 la usa.

FUNCIÓN	COMPARACIÓN O EFECTO	¿QUÉ DEBE PASAR EN EL LÍMITE EXACTO?	PRUEBA QUE LO COMPRUEBA

06 Copiar la plantilla

```
mkdir test/s08 (PowerShell, macOS, Linux)
cp andamiaje/s08/Recaudo.test.js test/s08/Recaudo.test.js
npx hardhat test test/s08/Recaudo.test.js
```

En `cmd` de Windows: `mkdir test\s08` y `copy andamiaje\s08\Recaudo.test.js test\s08\`. Si la carpeta ya existe, `mkdir` avisa y no pasa nada.

Salida esperada:

```
S08 · Recaudo · despliegue
  ✓ fija la meta y el dueño
S08 · Recaudo · aportar
  ✓ rechaza un aporte de cero

2 passing
```

La plantilla lee la variable de entorno `CONTRATO` para que el script de mutantes pueda correr la misma suite contra otro contrato. **No la toquen:** por defecto vale `Recaudo`.

07 Las herramientas que van a usar

CHULETA

```
// quién firma
const [beneficiario, ana, beto, carla] = await ethers.getSigners(); // en la fixture
await r.connect(ana).aportar({ value: eth(3) });

// el reloj de la red local (time viene de networkHelpers)
await time.increase(7 * 24 * 60 * 60); // avanza una semana y mina un bloque
await time.increaseTo(fin); // mina un bloque con marca fin: la SIGUIENTE tx cae en fin + 1
await time.setNextBlockTimestamp(fin); // la PRÓXIMA transacción cae exactamente en fin
await time.setNextBlockTimestamp(fin - 1n); // un segundo antes (fin es BigInt: 1n)

// reversiones: siempre con la razón exacta
await expect(tx).to.be.revertedWithCustomError(r, "PlazoVencido");
await expect(tx).to.be.revertedWithCustomError(r, "OwnableUnauthorizedAccount");
const F = await ethers.getContractFactory(CONTRATO); // constructor que revierte
await expect(F.deploy(0, PLAZO)).to.be.revertedWithCustomError(F, "ParametrosInvalidos");

// eventos y saldos
await expect(tx).to.emit(r, "Aporte").withArgs(ana.address, monto, total);
await expect(tx).to.changeEtherBalance(ethers, cuenta, monto); // ethers primero
await expect(tx).to.changeEtherBalances(ethers, [ana, r], [monto, -monto]);

// leer estado
expect(await r.totalRecaudado()).to.equal(eth(7));
expect(await r.estado()).to.equal(2n); // enum: 0n, 1n, 2n
```

TRES REGLAS DE HARDHAT 3 QUE ROMPEN TUTORIALES VIEJOS

1. La red se obtiene con `const { ethers, networkHelpers } = await network.create();`
2. `loadFixture` exige una función **con nombre** (`async function desplegar()`), no una flecha anónima.
3. No usen `.to.be.reverted` a secas: está desaconsejado y no dice **por qué** revirtió. Usen `revertedWithCustomError`.

DEPURAR: `console.log` DENTRO DEL CONTRATO

Si una prueba falla y no entienden por qué, pueden imprimir valores desde Solidity en la red local, **temporalmente**: `import "hardhat/console.sol";` y `console.log("total: %d", totalRecaudado);`. Aparece en la salida de `npx hardhat test`. Bórrenlo antes de desplegar, y no lo dejen en `Recaudo.sol`: el script de mutantes compara contra ese archivo.

08 Parte 1 - Escribir la suite

30 MINUTOS · ≥ 10 PRUEBAS

Mínimo 10 pruebas en total (los dos ejemplos cuentan), repartidas así:

GRUPO	MÍNIMO	QUÉ NO OLVIDAR
despliegue	2	Los parámetros inválidos del constructor, cada uno.
aportar	3	Suma al aportante y al total; evento con argumentos exactos; los límites del plazo.
recaudo exitoso	3	Permisos, repeticiones, saldo exacto que recibe el dueño, los límites de la meta.
recaudo fallido	3	Cada quien lo suyo; quien no aportó; el dueño no se lleva nada; repeticiones.

Ejemplo trabajado de una prueba de caso feliz, para que vean la forma completa (escribanla ustedes; es una de sus diez):

```
it("un aporte suma al aportante y al total, y emite Aporte", async () => {
  const { r, ana, beto } = await loadFixture(desplegar);
  await expect(r.connect(ana).aportar({ value: eth(2) }))
    .to.emit(r, "Aporte").withArgs(ana.address, eth(2), eth(2));
  await r.connect(beto).aportar({ value: eth(3) });
  expect(await r.aportes(ana.address)).to.equal(eth(2));
  expect(await r.totalRecaudado()).to.equal(eth(5));
});
```

Cómo leerla: arranca de la instantánea (`loadFixture`), Ana aporta y se exige el evento con los tres argumentos, Beto aporta, y se comprueba el estado guardado — no solo que no revirtió.

UNA PRUEBA FALLA POR LA RAZÓN CORRECTA, O NO PRUEBA NADA

Antes de dar por buena una prueba de reversión, pregúntense: **¿qué otra cosa podría hacerla revertir?** Si la transacción puede fallar por falta de fondos, por el plazo o por permisos además de por la regla que quieren probar, la prueba pasa aunque la regla esté mal escrita. Diseñen el escenario para que **solo** la regla pueda detenerla, y exijan el error exacto con `revertedWithCustomError`. Lo mismo al revés: una prueba de caso feliz debe comprobar el resultado (saldo, evento, estado), no solo que no revirtió.

Cómo sé que terminé esta parte: `npx hardhat test test/s08/Recaudo.test.js` muestra 10 o más `passing` y ningún `failing`.

09 Parte 2 · Medir la cobertura y leer el informe

10 MINUTOS · ≥ 80 %

```
npx hardhat test test/s08/Recaudo.test.js --coverage
```

Al final aparece una tabla con **todos** los contratos del repositorio. Con solo la plantilla (2 pruebas), la fila de `Recaudo` se ve así (salida real):

```
|| File Path | Line % | Statement % | Uncovered Lines
|| contracts\s08\Recaudo.sol | 11.54 | 15.79 | 49, 55-56, 60, 63-64, 66, 70-72, 74-75, ...
||
```

1. **Busquen su fila:** `contracts\s08\Recaudo.sol`. Las demás filas (y el `Total`) no importan: incluyen contratos de otras sesiones que su suite no toca. Los mutantes aparecen con 0 %: es lo esperado.
2. **Lean `Line %`**. Meta del curso: **80 o más**.
3. **Lean `Uncovered Lines`**. Cada número es una línea de `Recaudo.sol` que ninguna prueba ejecutó. Ábrala y pregúntense qué prueba la ejecutaría.
4. **Informe navegable:** Hardhat deja además `coverage/html/index.html`. Ábralo con el navegador: cada línea aparece en verde (ejecutada) o en rojo (nunca).

¿Y SI NO LLEGA A 100 %?

No hace falta. Las líneas `if (!ok) revert EnvioFallido();` solo se ejecutan si el receptor del pago lo rechaza, y eso exige un contrato auxiliar que las pruebas del laboratorio no usan. Una suite muy completa se queda en torno al 92 % por eso. Lo importante es que puedan explicar **por qué** cada línea queda sin cubrir: anótenlo en el informe.

Cómo sé que terminé esta parte: `Line %` ≥ 80 en la fila de `Recaudo.sol`. Hagan la captura.

10 Parte 3 · Cazar los mutantes

20 MINUTOS

10.1 · Correr el cazador

```
node scripts/s08/cazar-mutantes.js
```

El script corre su suite cuatro veces: contra `Recaudo` y contra cada mutante (cambia la variable `CONTRATO`). Funciona igual en PowerShell, cmd, macOS y Linux. Con solo la plantilla, la salida real es:

```
Suite: test/s08/Recaudo.test.js
```

```
✓ Recaudo (correcto)      2 pruebas en verde
```

```
X VIVO   RecaudoMutante1  2 pasan · 0 fallan
```

```
X VIVO   RecaudoMutante2  2 pasan · 0 fallan
```

```
X VIVO   RecaudoMutante3  2 pasan · 0 fallan
```

```
3 mutante(s) sobrevive(n). Hay un error que su suite no ve: piensen en los bordes.
```

Cómo leerla: un mutante está **MUERTO** si al menos una prueba falla contra él (su suite vio el error). Está **VIVO** si todas pasan (ese error llegaría a producción). Si el script dice que contra el contrato **correcto** falla alguna prueba, primero arreglen esa prueba: una suite que falla contra el contrato bueno no sirve para juzgar a nadie.

10.2 • El método, mientras haya vivos

Vuelvan a su inventario de límites (sección 5) y recorran, **para cada función**, las cinco preguntas de borde:

BORDE	LA PREGUNTA
Tiempo	¿Qué pasa un segundo antes, en el segundo exacto y un segundo después de cada plazo? (sección 7: <code>setNextBlockTimestamp</code>)
Umbral	¿Qué pasa con el valor exacto de cada límite: ni un wei más, ni uno menos?
Repetición	¿Qué pasa si la misma cuenta llama dos veces seguidas? ¿Qué impide la segunda: la regla, o algo más?
Cero y vacío	¿Qué pasa con 0, con la dirección cero?
Quién llama	¿Qué pasa si llama otra cuenta?

Cada vez que agreguen una prueba, corran otra vez el cazador. No hay límite de intentos: el script es la retroalimentación.

10.3 • Cómo sé que terminé

El script termina con `Los tres mutantes murieron: su suite detecta los tres errores.` Hagan la captura.

10.4 • Solo ahora: abrir los mutantes

Con los tres muertos, abran cada mutante y compárenlo con `Recaudo.sol` (en VS Code: clic derecho sobre los dos archivos → *Compare Selected*). Llenen esta tabla para el informe:

MUTANTE	¿CUÁL ERA EL ERROR?	¿QUÉ PRUEBA LO MATÓ?	¿POR QUÉ LA COBERTURA NO LO VEÍA?	¿QUÉ DAÑO HARÍA EN PRODUCCIÓN?
1				
2				
3				

11 Parte 4a · Desplegar con Ignition en la red local

5 MINUTOS

```
npx hardhat ignition deploy ignition/modules/Recaudo.js
```

Salida esperada (real, recortada):

```
You are running Hardhat Ignition against an in-process instance of Hardhat Network.
This will execute the deployment, but the results will be lost.
Hardhat Ignition 🚀
Deploying [ RecaudoModulo ]
Batch #1
  Executed RecaudoModulo#Recaudo
[ RecaudoModulo ] successfully deployed 🚀
Deployed Addresses
RecaudoModulo#Recaudo - 0x5FbDB2315678afecb367f032d93F642f64180aa3
```

Qué acaba de pasar: Ignition leyó el módulo, usó los parámetros por defecto (meta 10 ETH, una semana) y desplegó en una red en memoria que desaparece al terminar. **Cómo sé que funcionó:** `successfully deployed` y una dirección. Si falla aquí, fallará en Sepolia: se corrige aquí, sin gastar ETH de prueba.

POR QUÉ IGNITION Y NO UN SCRIPT SUELTO

Ignition registra cada paso en `ignition/deployments/chain-<id>/`. Si la conexión se corta a mitad del despliegue, al repetir el comando retoma donde quedó y no despliega dos veces. El mismo módulo sirve para cualquier red: cambian solo los parámetros (`m.getParameter`).

12 Parte 4b · Los tres secretos, en el keystore

10 MINUTOS · UNA VEZ

Para Sepolia hacen falta tres valores. Ninguno se escribe en un archivo del repositorio: van al **almacén cifrado de Hardhat** (keystore), que vive en su carpeta de usuario y se protege con una contraseña que ustedes eligen.

NOMBRE	DE DÓNDE SALE	POR QUÉ ES SECRETO
<code>SEPOLIA_RPC_URL</code>	Cuenta gratuita en un proveedor RPC (Alchemy o Infura): crear una app para <i>Ethereum Sepolia</i> y copiar la URL HTTPS.	La URL incluye su clave de API: otro podría gastar su cuota.
<code>SEPOLIA_PRIVATE_KEY</code>	La clave privada de la cuenta del curso en MetaMask (detalles de la cuenta → mostrar clave privada; pide la contraseña de MetaMask).	Quien la tenga controla la cuenta. Nunca la de una billetera con fondos reales.
<code>ETHERSCAN_API_KEY</code>	Cuenta gratuita en etherscan.io → <i>API Keys</i> → crear una.	Identifica sus peticiones de verificación.

```
npx hardhat keystore set SEPOLIA_RPC_URL
npx hardhat keystore set SEPOLIA_PRIVATE_KEY
npx hardhat keystore set ETHERSCAN_API_KEY
npx hardhat keystore list
```

Qué esperar: la primera vez, Hardhat pide **crear una contraseña** para el almacén (y confirmarla); después pide el valor de la variable, sin mostrarlo en pantalla. Las siguientes veces pide la contraseña que crearon. `keystore list` muestra los **nombres** guardados, nunca los valores. **Cómo sé que funcionó:** `list` muestra los tres nombres.

LO QUE NO SE HACE NUNCA

No pegar la clave privada ni la URL en `hardhat.config.js`, en un `.env`, en un chat ni en el informe. La **frase semilla** no va en ningún lado, tampoco en el keystore: Hardhat solo necesita la clave privada de una cuenta. Si una clave se filtra, se da por perdida: se crea una cuenta nueva y no se vuelve a usar.

13 Parte 4c • Desplegar y verificar en Sepolia

5 MINUTOS + ESPERA

Comprueben que la cuenta del curso tiene ETH de Sepolia (al menos 0,01 para desplegar y aportar). Después, en una sola línea:

```
npx hardhat ignition deploy ignition/modules/Recaudo.js --network sepolia --parameters
ignition/parametros/recaudo-sepolia.json --verify
```

1. Hardhat pide la **contraseña del keystore**: es la que crearon en la sección 12.
2. Si Ignition pide confirmar el despliegue en la red `sepolia`, lean el nombre de la red y confirmen.
3. Ignition envía la transacción y espera a que se incluya en un bloque: en Sepolia son decenas de segundos.
4. Al terminar muestra `successfully deployed` y la dirección `RecaudoModulo#Recaudo - 0x...`. **Anótenla.**
5. Con `--verify`, a continuación envía el código fuente al explorador e informa si la verificación tuvo éxito, con un enlace.

Cómo sé que funcionó: abran `https://sepolia.etherscan.io/address/<su dirección>`. En la pestaña *Contract* debe aparecer la marca de verificación y el código de `Recaudo.sol` legible.

SI LA VERIFICACIÓN FALLA

Esperen un minuto y repitan **exactamente el mismo comando**. Ignition sabe, por su registro en `ignition/deployments/chain-11155111/`, que el contrato ya existe: no lo despliega otra vez y solo reintenta la verificación. Si el error menciona la API key, revisen `keystore list`.

14 Parte 4d • Usar el contrato desde el explorador

1. En la página del contrato: *Contract* → *Write Contract* → *Connect to Web3*. Conecten MetaMask con la cuenta del curso, en Sepolia.

2. `aportar` : en el campo del valor (*payableAmount*, en ETH) escriban `0.005` y *Write*. Lean la ventana de la billetera antes de firmar: red Sepolia, contrato correcto, 0,005 ETH. Guarden el hash.
3. En *Read Contract*: `totalRecaudado` debe mostrar `5000000000000000`; `estado`, `0` (Activo) mientras no pasen los 30 minutos.
4. Pasados los 30 minutos, `estado` muestra `1` (Exitoso). Desde la cuenta que desplegó, `reclamarFondos`. Guarden el hash (opcional; puede hacerse en casa).

15 Antes de cualquier git push

```
git status
git diff --staged
```

SÍ SE SUBE

```
test/s08/Recaudo.test.js
ignition/deployments/chain-11155111/
(evidencia reproducible)
package-lock.json
```

NUNCA SE SUBE

Claves privadas, frases semilla, URLs de RPC con API key, archivos `.env`
`node_modules/`, `artifacts/`, `cache/`,
`coverage/`
Datos personales de cualquier tipo

El `.gitignore` del repositorio ya excluye lo generado. Aun así, **lean** la lista de `git status` antes de cada `push`: un secreto borrado en un commit posterior sigue en el historial.

16 Errores frecuentes

CASI TODO SE RESUELVE AQUÍ

SÍNTOMA	CAUSA Y SOLUCIÓN
«npx no se reconoce»	Node.js no está instalado o la terminal es anterior a la instalación. Node 22 LTS y terminal nueva.
No encuentra <code>hardhat.config.js</code> o «not inside a Hardhat project»	La terminal no está en <code>laboratorios-evm</code> . Usen <code>cd</code> .
El cazador dice «No se encontraron pruebas»	La suite no está en <code>test/s08/Recaudo.test.js</code> (revisen el nombre y la carpeta).
«Contra el contrato CORRECTO fallan N prueba(s)»	Una de sus pruebas espera algo que el contrato correcto no hace. Arréglenla antes de juzgar mutantes.
<code>loadFixture</code> falla con una función anónima	La fixture necesita nombre: <code>async function desplegar() { ... }</code> .
<code>ethers</code> no está definido, o <code>require is not defined</code>	Código de un tutorial de Hardhat 2. En Hardhat 3 se usa <code>import</code> y la red se obtiene con <code>const { ethers, networkHelpers } = await network.create()</code> .
Una prueba de plazo pasa contra todo	Nunca llega al segundo que quieren probar. Revisen qué hace cada función de <code>time</code> (sección 7).
Una prueba de reversión pasa, pero no por la regla	Algo más la hace revertir. Rediseñen el escenario (sección 8) y exijan el error exacto.
<code>changeEtherBalance</code> falla por una cantidad pequeña	Están comparando saldos a mano y restando mal el gas. Usen el matcher, que ya lo descuenta.
«TypeError: Cannot mix BigInt and other types»	<code>fin</code> y los montos son BigInt: <code>fin - 1n</code> , no <code>fin - 1</code> .
La cobertura muestra 0 % en <code>Recaudo.sol</code>	Corrieron otro archivo de pruebas, o <code>CONTRATO</code> quedó definido en la terminal. Cierren la terminal y abran otra.
El keystore no pide contraseña y falla el despliegue	La variable no está guardada: <code>npx hardhat keystore list</code> .
«insufficient funds» en Sepolia	La cuenta del curso no tiene ETH de prueba. Pidan al docente.
<code>--verify</code> falla	Falta <code>ETHERSCAN_API_KEY</code> , o el explorador aún no indexa el contrato: repetir el mismo comando tras un minuto.
Quieren desplegar otra vez en local con cambios	La red en memoria no guarda nada: basta repetir. En Sepolia, un segundo despliegue con otros parámetros necesita <code>--deployment-id</code> distinto.

17 Preguntas de reflexión

SE RESPONDEN CON SU SUITE Y SU SALIDA

1. ¿Qué cobertura tenía su suite la primera vez que superó el 80 %, y cuántos mutantes seguían vivos en ese momento? ¿Qué les dice eso sobre el requisito del 80 %?

2. Elijan una de las pruebas que mató a un mutante. Copien su código y expliquen línea por línea por qué **solo** la regla del contrato podía hacerla fallar.
3. ¿Qué líneas de `Recaudo.sol` quedaron sin cubrir en su informe final? Para cada una: ¿por qué, y qué prueba haría falta?
4. Escojan un mutante y describan un escenario concreto —quién, cuándo, cuánto— en el que ese error le cuesta dinero a una persona real.
5. ¿Qué hay en su carpeta `ignition/deployments/chain-11155111/` ? ¿Qué información permitiría a otra persona comprobar su despliegue?
6. La pregunta de la semana: si una suite con 88 % de cobertura dejó pasar tres errores, ¿qué evidencia le pedirían a un proveedor para creerle que su contrato está bien probado?

18 Lista de chequeo antes de entregar

- ☐ `test/s08/Recaudo.test.js` con ≥ 10 pruebas, todas en verde contra `Recaudo`.
- ☐ Captura de cobertura con `Line %` ≥ 80 en la fila de `Recaudo.sol`.
- ☐ Inventario de límites (sección 5) lleno.
- ☐ Captura del cazador con «Los tres mutantes murieron».
- ☐ Tabla de la sección 10.4 llena para los tres mutantes.
- ☐ Dirección de `Recaudo` en Sepolia con la marca de código verificado, desplegado con Ignition.
- ☐ Hash de al menos un aporte.
- ☐ Respuestas de reflexión.
- ☐ `git status` revisado: ningún secreto en el repositorio ni en el informe.

19 Rúbrica

LABORATORIO 08 · SEGUNDO CORTE

CRITERIO	PESO	EXCELENTE (5,0)	ACEPTABLE (3,5)	INSUFICIENTE (< 3,0)
Suite	25 %	≥ 10 pruebas de las cinco familias, nombres que se leen como especificación, razones exactas de reversión.	≥ 10 pruebas, casi todas de caso feliz.	Menos de 10, o alguna falla contra el correcto.
Cobertura	15 %	≥ 80 % y explicación de cada línea sin cubrir.	≥ 80 % sin explicación.	< 80 %.
Mutantes	35 %	Los tres muertos; tabla 10.4 completa y correcta; explican por qué la cobertura no los veía.	Los tres muertos, explicación superficial.	Algún mutante vivo.
Despliegue	25 %	Con Ignition, verificado, con aporte en cadena y ningún secreto en el repositorio.	Desplegado sin verificar.	Sin evidencia en cadena: no se califica.

20 Trabajo autónomo · el proyecto, en Hardhat

PARA LA SESIÓN 9

El plan pide llevar los contratos del proyecto a la estructura Hardhat. Estado mínimo del repositorio del equipo para la Sesión 9:

PIEZA	ESTADO MÍNIMO
Repositorio	En GitHub, los tres integrantes con permisos, el docente como colaborador. Estructura de la lámina C.1 (contracts, test, ignition, docs).
Versiones	Las del curso, exactas: Hardhat 3.16.0, Solidity 0.8.28, OpenZeppelin 5.6.1 (<code>npm install --save-exact</code>). <code>package-lock.json</code> incluido.
Contrato principal	Compila con <code>npx hardhat compile</code> . Aplica al menos dos patrones de la Sesión 7.
Suite inicial	Al menos 5 pruebas, con al menos una de cada familia (caso feliz, reversión, permisos, eventos, borde).
Secretos y <code>.gitignore</code>	Ninguno en el repositorio. El mismo <code>.gitignore</code> del curso.
README	Qué hace, cómo instalar y cómo correr las pruebas, en cinco líneas.

Además: traer el refactor de `CertificadosUSB` de la S7 con su suite corriendo en Hardhat, y crear cuenta en `ethernaut.openzeppelin.com` y resolver el nivel 0.

21 Referencias

- Hardhat 3 · *Configuration variables* y keystore. <https://hardhat.org/docs/learn-more/configuration-variables>
- Hardhat Ignition · *Verifying your deployment*. <https://hardhat.org/ignition/docs/guides/verify>
- OpenZeppelin Contracts 5.x · *Access Control*.
<https://docs.openzeppelin.com/contracts/5.x/api/access>
- ConsenSys · *How to Avoid Uploading Your Private Key to GitHub*. <https://consensys.io/blog/how-to-avoid-uploading-your-private-key-to-github-approaches-to-prevent-making-your-secrets-public>